JavaScript: The New Parts

In this tenth and final part of the series, we learn about Generators, the most exciting and significant feature of ES6. It builds on features, iterators and iterables covered in previous articles in this series. It is recommended that you review those before reading further.

Generators are not new to programming languages. They date back to 1974, when they were introduced in CLU language. Programming languages like Python have this feature with similar syntax. Generators have two interesting aspects. They look like functions, but behave like iterators. More importantly, they make asynchronous code look like synchronous code. In essence, they are an improvisation over callbacks and promises. If you are a programming language enthusiast, you might have heard about coroutines. Generators are a limited version of coroutines.

To get a full understanding of generators, let's look at them from three perspectives. The first is the control flow, the second is asynchronous programming and the third is the in-built capability that the ES6 runtime provides that makes programmers' lives easier (what I refer to as free gifts!).

The concept of generators involves a steep learning curve and might take time to grasp, but that is normal.

Control flow

JavaScript has run-to-completion semantics, which means the sequence of instructions is executed from first to last. Conditional and looping statements change the flow. Functions get executed from the first instruction to the last, except when a return or an exception is encountered inbetween. Generators introduce a mechanism where control is switched back and forth between the calling code and the generator function. The calling code is the controller function. A generator function has multiple entry points. It resumes from where it gave control to the calling function or instruction.

Generators syntax and flow

The program below introduces the syntax of generators. A function is made a generator by adding an '*' (asterisk) after the keyword function. The flow of the program breaks every time the keyword yield is encountered.

```
1.  function* HelloWorld() {
2.    console.log("Start");
3.    yield "hello";
4.    yield "world";
5.    console.log("End");
6.  }
```

We create an object *h* of the above generator and start interacting with it. As mentioned earlier, a generator gives control and takes it back using another piece of code. We can call it the generator controller. It could be in your main program or another function.

```
7. var h = HelloWorld();
8.  console.log(h.next());
9.  console.log(h.next());
        10. console.log(h.next());
```

You could execute the above program (lines 1 to 10), copying it to a file.

After instantiation of the generator object in Line 7, ensure that 'Start' gets printed only when the first token ("Hello") is fetched from Line 8. The function execution gets suspended at Line 3. After invoking another *next()*, control goes back to the generator function, which executes Line 4. The third invocation of the *next()* method brings you to the end of strings. So the 'End' string is printed and the end of the iterator is reached.

I strongly recommend that you to try this out in an interactive REPL or command line environment, copying Lines 1 to 7. I use the extension in the Chrome browser called Scratch JS.

```
> h.next()
Start
Object { value: "hello", done: false }
> h.next();
Object { value: "world", done: false }
```